MLOps Pipeline Report: Cats vs Dogs Binary Image Classification

Student Details

- **Name:** Shyam Gupta
- **ID:** 2024CS05012
- **Course:** MLOps (S2-25_AMLCSZG523)
- **Assignment 02:** End-to-End ML Model Development, CI/CD, and Production Deployment Experimental Learning (Binary image classification (Cats vs Dogs) for a pet adoption platform)
- **Total Marks:** 50

Submission Details

Artifact	Location
Public GitHub repository	https://github.com/2024CS05012/MLOps-Cat-Dog-Classification
Demo video (Youtube)	https://youtu.be/pAf7N5pxf30
CI/CD workflow	https://github.com/2024CS05012/MLOps-Cat-Dog-Classification/actions

---

1. Project Overview

This project implements an end-to-end MLOps pipeline for a binary image classification use case: identifying whether an uploaded pet image is a cat or a dog. The solution is designed for a pet adoption platform where model predictions can support image tagging or listing verification.

The pipeline covers dataset preparation, model training, experiment tracking, artifact versioning, API packaging, containerization, CI/CD automation, deployment, smoke testing, and basic post-deployment monitoring.

2. Use Case and Dataset

The selected use case is binary image classification for cats and dogs.

Dataset:

- Source: Kaggle Cats and Dogs classification dataset
- Classes: `cat` and `dog`
- Raw data location: `data/raw/cat` and `data/raw/dog`
- DVC metadata: `data/raw/cat.dvc` and `data/raw/dog.dvc`

Preprocessing:

- Images are loaded as RGB.

- EXIF orientation is corrected.
- Images are resized to `224x224`.
- Data is split into train, validation, and test sets using an `80% / 10% / 10%` split.
- Processed data is stored under `data/processed`.

Detailed preprocessing flow:

1. `load_rgb_image()` opens each image, applies EXIF orientation correction using `ImageOps.exif_transpose()`, and converts the image to RGB.
2. `resize_image()` resizes every image to `224x224` using bilinear interpolation.
3. `preprocess_image_file()` creates the required output folder, applies loading/resizing, and saves the processed file as a JPEG with quality `95`.
4. `split_files()` uses a fixed random seed, `42`, so the train/validation/test split is reproducible.
5. `preprocess_dataset()` processes both classes and writes files into class-specific folders under each split.

Processed folder structure:

```
data/processed/
├── train/
│   ├── cat/
│   └── dog/
├── val/
│   ├── cat/
│   └── dog/
└── test/
    ├── cat/
    └── dog/
```

Data augmentation:

- Training data uses random horizontal flip.
- Training data uses random rotation.
- Training data uses color jitter.
- Validation and test data use deterministic transforms only.

Exact training transforms:

```
RandomHorizontalFlip()
RandomRotation(10)
ColorJitter(brightness=0.1, contrast=0.1)
Resize((224, 224))
ToTensor()
Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
```

Validation and test transforms:

```
Resize((224, 224))
ToTensor()
Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
```

Relevant files:

- `src/data/preprocess.py`
- `src/data/dataset.py`
- `scripts/preprocess_data.py`
- `dvc.yaml`
- `dvc.lock`

3. Tools and Technologies

The project uses the following open-source tools:

Area	Tool
Source code versioning	Git
Data and artifact versioning	DVC
Model framework	PyTorch
Experiment tracking	MLflow
API service	FastAPI
Containerization	Docker
CI/CD	GitHub Actions
Deployment target	Docker Compose
Optional deployment manifests	Kubernetes
Testing	Pytest
Monitoring/logging	Python logging and in-app counters

4. M1: Model Development and Experiment Tracking

4.1 Data and Code Versioning

Source code is versioned using Git. The repository includes project source code, tests, scripts, Docker configuration, deployment manifests, CI/CD workflows, and setup documentation.

DVC is used to version the dataset and pipeline artifacts. The raw cat and dog folders are tracked using:

- `data/raw/cat.dvc`
- `data/raw/dog.dvc`

The DVC pipeline is defined in `dvc.yaml` and locked in `dvc.lock`.

DVC stages:

Stage	Purpose
`preprocess`	Converts raw images into processed train/validation/test folders
`train`	Trains the baseline CNN and generates model/plot artifacts

The `preprocess` stage depends on:

- `data/raw`
- `scripts/preprocess_data.py`
- `src/data/preprocess.py`

The `preprocess` stage outputs:

- `data/processed`

The `train` stage depends on:

- `data/processed`
- `scripts/train_model.py`
- `src/models/train.py`
- `src/models/model.py`

The `train` stage outputs:

- `artifacts/models/best_model.pt`
- `artifacts/models/latest_model.pt`
- `artifacts/plots/loss_curve.png`
- `artifacts/plots/accuracy_curve.png`
- `artifacts/plots/confusion_matrix.png`

The lightweight DVC training command is:

```
python3 scripts/train_model.py --epochs 1 --batch-size 8 --max-train-
samples 256 --max-eval-samples 64
```

This keeps CI/CD execution practical while still demonstrating the full reproducible pipeline.

4.2 Model Building

The project implements a baseline convolutional neural network using PyTorch.

Model file:

- `src/models/model.py`

Training logic:

- `src/models/train.py`

- `scripts/train_model.py`

Model architecture:

Layer block	Details
Convolution block 1	`Conv2d(3, 16, kernel_size=3, padding=1)`, ReLU, MaxPool
Convolution block 2	`Conv2d(16, 32, kernel_size=3, padding=1)`, ReLU, MaxPool
Convolution block 3	`Conv2d(32, 64, kernel_size=3, padding=1)`, ReLU, MaxPool
Classifier	Flatten, Dropout `0.3`, Linear, ReLU, Linear to 2 classes

Training details:

- Optimizer: Adam
- Loss function: CrossEntropyLoss
- Best model selection: validation accuracy
- Final evaluation: test loss and test accuracy
- Dataset loader: `torchvision.datasets.ImageFolder`

Saved model artifacts:

- `artifacts/models/best_model.pt`
- `artifacts/models/latest_model.pt`

The model is serialized using PyTorch checkpoint format (`.pt`). The checkpoint stores:

- model state dictionary
- class names

The model artifact is loaded during API startup from:

```
artifacts/models/best_model.pt
```

4.3 Experiment Tracking

MLflow is used for experiment tracking.

The training process logs:

- epochs
- batch size
- learning rate
- train loss
- validation loss
- train accuracy
- validation accuracy
- test loss
- test accuracy

- model checkpoint
- loss curve
- accuracy curve
- confusion matrix

MLflow experiment name:

```
cats-dogs-classification
```

MLflow is used inside `train_model()` with `mlflow.start_run()`, `mlflow.log_params()`, `mlflow.log_metrics()`, `mlflow.log_artifact()`, and `mlflow.log_artifacts()`.

Generated visual artifacts:

- `artifacts/plots/loss_curve.png`
- `artifacts/plots/accuracy_curve.png`
- `artifacts/plots/confusion_matrix.png`

5. M2: Model Packaging and Containerization

5.1 Inference Service

The trained model is wrapped in a FastAPI REST API.

Main API file:

- `src/api/main.py`

Inference utility:

- `src/api/inference.py`

API endpoints:

Endpoint	Method	Purpose
`/health`	GET	Returns service status and whether model is loaded
`/predict`	POST	Accepts an uploaded image and returns predicted label and class probabilities
`/metrics`	GET	Returns request count and average latency

The prediction endpoint accepts an image file and returns a response containing:

- predicted label
- probability for `cat`
- probability for `dog`

API startup behavior:

- On startup, the API attempts to load `artifacts/models/best_model.pt`.
- If the model is available, `/health` returns `model_loaded: true`.
- If the model file is missing, the API still starts, `/health` remains available, and `/predict` returns HTTP `503`.

Prediction flow:

1. The uploaded image is read as bytes.
2. PIL opens the image.
3. The image is converted to RGB.
4. The inference transform resizes and normalizes it.
5. The PyTorch model runs in `torch.no_grad()` mode.
6. Softmax converts logits into class probabilities.
7. The label with the highest probability is returned.

Example health response:

```
{
  "status": "ok",
  "model_loaded": true
}
```

Example prediction response:

```
{
  "label": "cat",
  "probabilities": {
    "cat": 0.56,
    "dog": 0.44
  }
}
```

5.2 Environment Specification

The project uses pinned dependencies for reproducibility.

Two dependency files are provided:

File	Purpose
`requirements.txt`	Full development, training, testing, DVC, MLflow, and CI environment
`requirements-api.txt`	Smaller inference-only environment for the Docker image

This separation keeps the deployed container smaller while preserving the full training and experiment environment for development and CI.

5.3 Containerization

The inference service is containerized using Docker.

Docker file:

- `Dockerfile`

The Docker image:

- uses `python:3.11-slim`
- installs API dependencies from `requirements-api.txt`
- copies API source code and monitoring helpers
- copies model artifacts from `artifacts/models`
- verifies that `artifacts/models/best_model.pt` exists
- starts the FastAPI service with Uvicorn

The Dockerfile intentionally installs `requirements-api.txt` instead of the full `requirements.txt`. This avoids including training-only tools such as MLflow, DVC, Kaggle utilities, testing packages, and plotting packages inside the runtime image.

Example local commands:

```
docker build -t cats-dogs-mlops:latest .
docker run -p 8000:8000 cats-dogs-mlops:latest
curl http://localhost:8000/health
curl -X POST http://localhost:8000/predict -F "file=@data/sample/cat.jpg"
```

6. M3: CI Pipeline for Build, Test, and Image Creation

6.1 Automated Testing

Automated tests are implemented using Pytest.

Test files:

- `tests/test_preprocess.py`
- `tests/test_inference.py`
- `tests/test_api.py`
- `tests/test_dataset_download.py`
- `tests/test_simulate_requests.py`

The tests cover:

- image preprocessing and resizing
- deterministic train/validation/test splitting
- inference utility output format
- API health endpoint
- API behavior when model artifact is unavailable
- simulated post-deployment request generation

Specific test examples:

- `test_preprocess_image_file_resizes_and_converts_rgb` verifies that a grayscale input image becomes RGB and is resized to `224x224`.
- `test_split_files_is_deterministic` verifies that the split is reproducible and follows the expected 8/1/1 split for 10 files.
- `test_predict_image_returns_label_and_probabilities` verifies that inference returns a valid class label and probabilities summing to 1.
- `test_health_endpoint` verifies the API health endpoint.
- `test_predict_without_model_returns_503` verifies graceful behavior when the model artifact is missing.

Local test result:

```
7 passed
```

6.2 CI Setup

GitHub Actions is used for CI.

Workflow file:

- `.github/workflows/ci.yml`

The CI workflow runs on:

- push to any branch
- pull request

CI steps:

1. Checks out the repository.
2. Sets up Python 3.11.
3. Installs dependencies from `requirements.txt`.
4. Restores artifacts from DVC or rebuilds them using Kaggle credentials.
5. Verifies that `artifacts/models/best_model.pt` exists.
6. Runs unit tests with Pytest.
7. Builds the Docker image.
8. Pushes the image to GitHub Container Registry on push events.

CI artifact restoration strategy:

- First, the workflow tries `dvc pull`.
- If a DVC remote is configured through `DVC_REMOTE_URL`, it uses that remote.
- If DVC pull fails, the workflow checks for `KAGGLE_USERNAME` and `KAGGLE_KEY`.
- With Kaggle credentials, it downloads the dataset and runs `dvc repro`.
- If neither DVC remote nor Kaggle credentials are available, CI fails clearly because the Docker image requires `artifacts/models/best_model.pt`.

This makes the pipeline reproducible even when large artifacts are not committed to Git.

6.3 Artifact Publishing

The CI pipeline publishes Docker images to GitHub Container Registry.

Image tags:

- commit SHA tag
- `latest` tag

The workflow lowercases the GitHub repository name before creating the GHCR image path.

7. M4: CD Pipeline and Deployment

7.1 Deployment Target

The main deployment target is Docker Compose.

Deployment file:

- `deployment/docker-compose.yml`

The Compose deployment:

- runs the API container
- exposes port `8000`
- supports using the image published by CI/CD through the `IMAGE_NAME` environment variable

Docker Compose deployment command:

```
docker compose -f deployment/docker-compose.yml up -d --no-build
```

In CD, the image is pulled from GHCR first, and Compose starts that exact image.

Kubernetes manifests are also included as an optional deployment target:

- `deployment/k8s/deployment.yaml`
- `deployment/k8s/service.yaml`

The Kubernetes deployment includes:

- one API replica
- container port `8000`
- readiness probe on `/health`
- liveness probe on `/health`
- LoadBalancer service mapping port `80` to container port `8000`

7.2 CD Flow

GitHub Actions is used for CD.

Workflow file:

- `.github/workflows/cd.yml`

The CD workflow is triggered when the CI workflow completes successfully on the `main` branch.

CD steps:

1. Checks out the repository at the CI commit SHA.
2. Sets up Python.
3. Installs smoke test dependencies.
4. Restores DVC metadata or artifacts for smoke inputs.
5. Builds the deployment image name.
6. Logs in to GitHub Container Registry.
7. Pulls the latest Docker image.
8. Deploys the service using Docker Compose.
9. Waits for the API to start.
10. Runs smoke tests.
11. Collects post-deployment prediction results.
12. Uploads the post-deployment report as a workflow artifact.

7.3 Smoke Tests

Smoke testing is implemented in:

- `scripts/smoke_test.py`

The smoke test:

- calls `/health`
- sends one image to `/predict`
- fails if either request fails

If `data/sample/cat.jpg` is not available, the smoke test creates a synthetic `224x224` RGB JPEG image and uses that for prediction. This ensures the deployment validation can still run in CI/CD even when sample images are absent.

Example command:

```
python scripts/smoke_test.py --base-url http://localhost:8000
```

8. M5: Monitoring, Logs, and Post-Deployment Tracking

8.1 Basic Monitoring and Logging

The FastAPI service includes request logging middleware.

Relevant files:

- `src/api/main.py`
- `monitoring/logging_config.py`

- `monitoring/metrics.py`

Logged request details:

- request path
- HTTP method
- status code
- latency in milliseconds

Prediction logs include:

- uploaded filename
- predicted label

The API avoids logging raw image content.

The service also exposes basic runtime metrics through:

- `/metrics`

Metrics include:

- request count
- average latency in seconds

8.2 Model Performance Tracking After Deployment

Post-deployment request simulation is implemented in:

- `scripts/simulate_requests.py`

The script sends a small labeled batch to the deployed API and records:

- image name
- true label
- predicted label
- class probabilities

If real sample images exist in `data/sample`, the script uses those files and infers the true label from the filename. If no sample images are present, it creates two synthetic labeled images:

- `simulated-cat.jpg` with true label `cat`
- `simulated-dog.jpg` with true label `dog`

Example report:

- `artifacts/reports/post_deploy_predictions.example.json`

The CD workflow uploads the generated post-deployment prediction report as a GitHub Actions artifact named:

- `post-deployment-predictions`

9. Reproducibility Instructions

Create and activate the environment:

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Download the Kaggle dataset:

```
python scripts/download_kaggle_dataset.py \
  --dataset-name bhavikjikadara/dog-and-cat-classification-dataset \
  --output-dir data/raw
```

Run the DVC pipeline:

```
dvc repro
```

Run only preprocessing:

```
python scripts/preprocess_data.py
```

Run only model training:

```
python scripts/train_model.py --epochs 3 --batch-size 16
```

Open MLflow locally:

```
mlflow ui
```

Run tests:

```
pytest -q
```

Run the API locally:

```
uvicorn src.api.main:app --host 0.0.0.0 --port 8000
```

Check health:

```
curl http://localhost:8000/health
```

Make a prediction:

```
curl -X POST http://localhost:8000/predict -F "file=@data/raw/cat/9733.jpg"
```

View basic metrics:

```
curl http://localhost:8000/metrics
```

Run smoke test:

```
python scripts/smoke_test.py --base-url http://localhost:8000
```

Run simulated post-deployment tracking:

```
python scripts/simulate_requests.py \
  --base-url http://localhost:8000 \
  --output artifacts/reports/post_deploy_predictions.json
```

10. Assignment Coverage Summary

Assignment Requirement	Status	Evidence
Git source code versioning	Covered	Repository source files and workflows
DVC dataset/preprocessed data tracking	Covered	`data/raw/*.dvc`, `dvc.yaml`, `dvc.lock`
Baseline model	Covered	`src/models/model.py`
Serialized trained model	Covered	`artifacts/models/best_model.pt`
Experiment tracking	Covered	MLflow logging in `src/models/train.py`
Confusion matrix and loss curves	Covered	`artifacts/plots`
REST inference service	Covered	FastAPI app in `src/api/main.py`
Health endpoint	Covered	`/health`
Prediction endpoint	Covered	`/predict`

Assignment Requirement	Status	Evidence
Pinned dependencies	Covered	`requirements.txt`, `requirements-api.txt`
Dockerfile	Covered	`Dockerfile`
Unit tests	Covered	`tests/`
CI pipeline	Covered	`.github/workflows/ci.yml`
Docker image publishing	Covered	GHCR push in CI workflow
Deployment manifests	Covered	Docker Compose and Kubernetes YAML
CD pipeline	Covered	`.github/workflows/cd.yml`
Smoke test	Covered	`scripts/smoke_test.py`
Request/response logging	Covered	API middleware logging
Basic metrics	Covered	`/metrics` endpoint
Post-deployment prediction tracking	Covered	`scripts/simulate_requests.py`
ZIP submission	Covered	Final packaging step
Screen recording	Covered	Final demonstration step

11. Conclusion

The project satisfies the core requirements of the assignment by implementing a complete MLOps workflow for binary image classification. It includes reproducible preprocessing and training, experiment tracking, containerized inference, automated testing, CI image creation and publishing, CD deployment, smoke testing, logging, basic metrics, and post-deployment prediction tracking.